
How this website *works.*

The complete technical context for workingwithgod.live — the architecture, the deployment process, the constraints, and how to troubleshoot it.

Written to be given to an AI assistant. Paste it, attach it, or keep it in the project folder. Everything an assistant needs to help safely is in here.

READ FIRST

Standing instruction for the assistant

If you are an AI assistant helping with this website, these rules apply to every single request. They are not suggestions.

1. Every code change must end with the full push sequence. No exceptions.

Never leave an edited file sitting on disk with no instructions. Whenever you change any file in this project, finish your reply with the complete, copy-pasteable command block below, filled in with the actual files you changed. Do not abbreviate it, do not assume the person remembers it, and do not say “then commit and push as usual.”

```
cd C:\Users\User\Desktop\wmg-site-v6

git add <the exact files you changed>
git commit -m "<a short description of the change>"
git push <remote> main
```

The person following this is not a developer. A missing step means a change that never reaches the website, which is the single most common failure on this project.

2. The commands only work inside the project folder. Say so every time.

A terminal window does **not** open in the project. It opens wherever Windows feels like — usually `C:\WINDOWS\system32`. Every git command run from there fails with “*fatal: not a git repository*”, which looks alarming and means nothing except “you are standing in the wrong room”.

This has already cost time twice. So whenever you give commands, **always include the `cd` line first**, and tell the person to check the prompt before continuing.

THE EASY WAY — NO TYPING PATHS

1. Open the folder in File Explorer:

`C:\Users\User\Desktop\wvg-site-v6`

2. Click the address bar at the top (where the folder path is shown)

3. Type `cmd` and press Enter

A terminal opens already inside the folder. No `cd` needed.

OR TYPE IT YOURSELF

```
cd C:\Users\User\Desktop\wvg-site-v6
```

Before running anything else, check the prompt. It must end with `wvg-site-v6>`. If it still says `system32>`, the folder change did not happen and every command after it will fail.

Each new terminal window starts over. Command Prompt and PowerShell do not share a location either — so this step is repeated every session, not done once.

3. Check the remote name before every push. It is different on every machine.

Only one repository deploys the live site:

```
github.com/EliyahuWWG/WWG-Code    branch: main
```

But the short *nickname* git uses for it is a local label, set per machine, and it is **not the same everywhere**. Never assume it. Run this first:

```
git remote -v
```

Read the output and use whichever name sits beside `EliyahuWWG/WWG-Code`.

ON THIS MACHINE	THE NAME IS USUALLY
Eliyahu's laptop <i>cloned from GitHub</i>	<code>origin</code> — so the command is <code>git push origin main</code>
EchoPulse's machine	<code>client</code> — because <code>origin</code> there points at a <i>different</i> , unrelated repository (<code>shauryak890/reframedrealityreact</code>) that deploys nothing

Both mistakes have already happened: a push to the wrong `origin` that reported success and changed nothing, and a `git push <remote> main` on a machine where no remote called `client` existed. One `git remote -v` prevents both.

4. Batch changes. One deploy, not five.

Each push triggers a production build costing **15 credits of a 300/month allowance**. If several changes are being discussed, collect them and push once. Never advise a push after every individual edit.

5. If you cannot push, say so plainly and hand over the commands

Assistants running in a sandbox generally *cannot* push to this repository — the GitHub proxy returns a 403 because the repo is not in the session's authorised set. This is not something more context or a retry will fix. Do not attempt it repeatedly. State it once, then give the person the exact commands to run themselves.

What talks to what

Five separate services are involved. Knowing which one owns which problem is most of troubleshooting.

SERVICE	WHAT IT DOES	WHO OWNS IT
GitHub EliyahuWWG/WWG-Code	Holds the source code. A push to <code>main</code> is what triggers a deployment.	Eliyahu
Netlify project workingwithgod	Builds the site from GitHub and serves it to visitors. Also issues the HTTPS certificate.	Eliyahu
GoDaddy	Owens the domain name and holds the DNS records that point it at Netlify. Also hosts the Microsoft email.	Eliyahu
Google Apps Script + Google Sheet	Receives form submissions, writes them to the spreadsheet, emails Eliyahu. Also serves the editable site content.	Eliyahu
Microsoft 365	Delivers <code>eliyahu@workingwithgod.live</code> . Entirely separate from the website despite sharing the GoDaddy DNS page.	Eliyahu

What happens when someone visits the site

```
visitor types workingwithgod.live
→ DNS at GoDaddy answers: 75.2.60.5
→ that is Netlify, which serves the prerendered pages
→ done. GitHub is not involved at page-view time.
```

What happens when someone fills in a form

visitor submits a form

- the browser posts DIRECTLY to the Google Apps Script web app
- the script writes a row to the spreadsheet
- the script emails Eliyahu the full submission
- the browser reads the reply and shows success or failure

The visitor learns within about two seconds whether it worked. That is deliberate — the person who submitted is the only one who still has the data if something fails.

Never reintroduce Netlify's outgoing webhook for forms

An earlier version routed submissions through a Netlify webhook to the Apps Script. It cannot work: Apps Script answers every POST with a 302 redirect it cannot be told not to send, Netlify counts any non-2xx as a failed delivery, and Netlify silently disables a webhook after roughly six failures.

So the webhook died *while working perfectly*, with no alert and no delivery log, and registrations stopped reaching the spreadsheet for weeks. If anyone suggests wiring it back up, this is why not.

Content Eliyahu changes himself — no code, no deploy

The **sponsor of the month** and the **next roundtable date** live in the “**Site content**” tab of the registrations spreadsheet. He edits a cell and the page picks it up within five minutes.

Never put these in code. A deploy costs credits; a cell edit costs nothing. The values in `src/data.js` are only the fallback shown if the spreadsheet cannot be reached.

Deploying a change

The whole process, every time, with nothing assumed.

- 01 Open a terminal and go to the project folder.** A new terminal window always starts somewhere else, so this step is never optional.

```
cd C:\Users\User\Desktop\wmg-site-v6
```

The prompt must now show the project path. If it still shows `C:\WINDOWS\system32`, the folder change did not happen and every command after this will fail with “not a git repository”.

- 02 See what has actually changed,** before committing anything.

```
git status
```

It lists every file that differs from what is published. Read it. If a file you did not expect is in that list, stop and ask — do not commit it just because it is there.

This repository often shows a dozen files as “modified” purely because of Windows line endings, with no real change inside them. That is why the next step names files explicitly rather than adding everything.

- 03 Stage only the files that were actually changed.** Naming them explicitly avoids sweeping in unrelated edits.

```
git add src/data.js src/pages/About.jsx
```

Avoid `git add -A` unless you have checked what it will include. This repository regularly shows dozens of files as “modified” purely because of Windows line endings, with no real change in them.

- 04 Commit with a description of what changed.**

```
git commit -m "copy: reword the About page opening"
```

- 05 Push to the client remote.**

```
git push <remote> main
```

Netlify sees the push and starts building automatically. No Netlify command is needed.

06 Watch the build. Netlify > **Deploys**. It takes about two minutes.

Green means live. Red means the build failed — in which case *nothing* is published, the previous version keeps serving, and no credits are charged. Open the log, fix, and push again.

If git says “index.lock: File exists”

A previous command was interrupted and left a marker file behind. No git process is actually running. Delete it and retry:

```
del .git\index.lock
```

If the push is rejected as “behind”

Someone else pushed first. Pull their work in, then push again:

```
git pull client main  
git push <remote> main
```

If the same file was edited on both sides, git will report a conflict. Do not guess — send the conflict to EchoPulse.

PART 03

How Netlify works, and its one hard limit

Netlify is on a credit-based plan with a ceiling that can take the whole site offline. This is the most important operational constraint on the project.

300 credits a month. A deploy costs 15. That is twenty deploys.

The free plan has a **hard cap** — it cannot be exceeded and cannot be topped up. When the credits run out, **every project on the account is paused** and visitors see a “Site not available” page until the next billing month begins.

Because of this, batching changes is not tidiness, it is the difference between a working site and a paused one. Ten separate pushes for ten copy tweaks is 150 credits — half the month’s allowance for work that could have cost 15.

WHAT USES CREDITS	RATE
A production deploy (push to <code>main</code>)	15 credits
Bandwidth	20 credits per GB
Web requests	2 credits per 10,000
Deploy previews and branch deploys	Free
Failed builds	Free — nothing is published or charged
Form submissions	Free and unlimited

At this site's traffic, visitors barely register. Deploys are essentially the entire bill. Check the remaining balance on the Netlify project overview before planning work.

The domain

DNS RECORDS AT GODADDY, AS CONFIGURED

```
A      @      75.2.60.5
CNAME  www    workingwithgod.netlify.app
```

The domain stays registered at GoDaddy; only these two records point it at Netlify. Netlify issues and renews the HTTPS certificate automatically once the domain resolves to it.

Troubleshooting

Ordered by how often each one actually happens.

The change did not appear on the website

CHECK	WHAT IT MEANS
Was it pushed to the repository that actually deploys?	The most common cause by far. Run <code>git remote -v</code> and confirm you pushed to whichever name sits beside <code>EliyahuWWG/WWG-Code</code> . A push to any other remote reports success and changes nothing.
Did the commit actually include the file?	<code>git show --stat HEAD</code> . If the file is not listed, it was never staged.
Did the Netlify build go red?	Netlify > Deploys. A failed build publishes nothing and leaves the old version live, which looks exactly like “nothing happened”.
Is the browser showing a cached copy?	Try a private/incognito window before concluding anything.

The site shows a certificate or “not private” warning

WHAT YOU SEE	WHAT TO DO
<code>ERR_CERT_COMMON_NAME_INVALID</code>	Netlify has not issued the certificate for the custom domain yet. Netlify > Domain management > HTTPS > Verify DNS configuration , then Provision certificate .
“We could not provision a Let’s Encrypt certificate”	Check the real state via the API rather than trusting the banner, which goes stale: <code>GET api.netlify.com/api/v1/sites/workingwithgod.netlify.app/ssl</code> with a personal access token. If it returns <code>"state":"issued"</code> , the certificate exists and the banner is wrong.
The button does nothing, repeatedly	Let’s Encrypt allows 5 failed validations per hostname per hour , refilling one slot every 12 minutes. Clicking repeatedly keeps the bucket empty and guarantees rejection. Stop for a full hour, then try <i>once</i> .
“You cannot visit this site because it uses HSTS”	A security policy cached in the browser from the site’s previous host. Clear it at <code>chrome://net-internals/#hsts</code> > Delete domain security policies. It cannot block certificate issuance — it only affects your own browser.

A form submission did not arrive

CHECK	WHAT IT MEANS
Run <code>diagnose</code> in the Apps Script editor	It checks every link in the chain in order and names the first broken one in plain English. Always start here.
The Status tab of the spreadsheet	Every submission and every daily check is logged there. A last entry that is days old means the script has stopped running.
Was the web app redeployed as a <i>new version</i> ?	Apps Script › Deploy › Manage deployments › pencil › New version . Choosing “New deployment” instead creates a different address and leaves the old code running, so changes appear to do nothing.
Is “Who has access” set to Anyone ?	Not “Anyone with a Google Account”. That setting makes the endpoint ask visitors to sign in, and submissions silently fail.

The build failed

Open the failing deploy in Netlify and read the last 20 lines of the log — the real error is near the bottom, not the top. Send that section to EchoPulse; it names the file and usually the line.

Nothing is published by a failed build. The site keeps serving the previous version, so there is no urgency and no need to rush a fix.

Conventions that will catch you out

Deliberate choices in this codebase that look like bugs if you do not know about them.

THING	WHAT TO KNOW
<code></code> renders bold , not italic	Deliberate, set in <code>src/index.css</code> . Marking a book title with <code></code> makes it look like shouting.
<code>.work-title</code>	The class to use for the title of a work. Italic, inherits the surrounding weight.
<code>btn-onink</code> vs <code>btn-solid</code>	<code>btn-onink</code> is a light button for dark sections; <code>btn-solid</code> is dark for light sections. Swapping them makes a button invisible.
Line breaks in headings	Controlled with a non-breaking space, not <code>
</code> , so they stay correct at every screen width.
The <code>CTA</code> component	Its default text appears on Home, About, Services and the blog. Speaking, Events and The Book pass their own. Changing the default changes five pages at once.

Do not touch these without asking EchoPulse

- **DNS records at GoDaddy.** Two rows point the domain at Netlify. Every other row on that page is the Microsoft email. One wrong deletion takes the mail down, and mail is harder to restore than a website.
- **Nameservers.** Never. Changing them hands over the entire domain, email included, in one action.
- **The GoDaddy “Websites + Marketing” subscription.** Cancelling it deletes the old website immediately — not at the end of the paid month. It is the only fallback if anything goes wrong with the new site.
- **The Apps Script web app URL.** Redeploying it as a “new deployment” rather than a “new version” breaks every form on the site silently.

NOTE

A note on who does what

Eliyahu should not be hand-editing code, and an assistant should not encourage it

GitHub's web editor will happily accept a broken tag and commit it straight to the live site. There is no build check in that editor, no undo once it is pushed, and every commit costs a deploy from a very limited monthly allowance.

It has already come close to going wrong: an assistant proposed a copy fix that would have published a sentence with a closing quotation mark and no opening one, and a comma in the wrong place, directly to the live About page.

The better arrangement, and the one this project runs on: Eliyahu spots what needs changing and says so; EchoPulse makes the change, verifies the build, and pushes. He is doing the valuable half already. The typing should not be his job, and neither should the risk.

If an assistant is asked to make a code change here, the right output is a clear explanation plus the exact commands — handed to whoever is going to run them — not an invitation to edit production by hand.

What Eliyahu can safely change himself, any time

- The **sponsor of the month** and the **next roundtable date** — the "Site content" tab of the registrations spreadsheet. Live within five minutes, no deploy, no cost, nothing to break.
- Anything in the spreadsheet itself — adding columns, notes and formatting is safe; the script matches columns by name, not position.

Questions at any point, send a screenshot rather than guessing.

EchoPulse